

- Informe Técnico: Modelado, Evaluación y Despliegue - Clasificación (Fases 4-6 CRISP-DM)
 - 1. Fase: Modelamiento (Modeling)
 - 1.1. Estrategia de Validación y Prevención de Data Leakage (Fuga de Datos)
 - 1.2. Selección de Variables y Decisión de Ingeniería de Cambio de Target
 - 1.3. Algoritmos Entrenados
 - 2. Fase: Evaluación (Evaluation)
 - 2.1. Resultados del Rendimiento en el Conjunto de Prueba (Test Set)
 - 2.2. Análisis de Resultados e Hipótesis del Dataset Sintético
 - 2.3. Validación Cruzada (Cross-Validation) para Descartar Sobreajuste
 - 3. Fase: Despliegue (Deployment)

Informe Técnico: Modelado, Evaluación y Despliegue - Clasificación (Fases 4-6 CRISP-DM)

Autor: Héctor Aguila

Asignatura: Fundamentos de Machine Learning

Metodología: CRISP-DM

1. Fase: Modelamiento (Modeling)

1.1. Estrategia de Validación y Prevención de Data Leakage (Fuga de Datos)

Para garantizar que los modelos se evalúen de manera justa y sin sesgos, se implementó una división rigurosa de los datos:

- **División Temprana (Train/Test Split):** Se separó el dataset en un **80% para entrenamiento (2233 registros)** y un **20% para prueba (559 registros)** *antes* de realizar cualquier procesamiento o escalado de variables. Esto asegura que el conjunto de prueba actúe como "datos del futuro" reales y desconocidos.

- **Encapsulamiento en Pipelines:** En lugar de estandarizar todo el dataset junto, se estructuró el preprocesamiento (como el escalado con `StandardScaler`) dentro de un `Pipeline` de `scikit-learn`. El pipeline calcula la media y la desviación estándar **únicamente en el conjunto de entrenamiento (Train)** y luego aplica esa misma escala al de prueba (Test), evitando que información de test se filtre al entrenamiento (mitigando el *Data Leakage*).
- **Semilla y Estratificación:** Se utilizó `random_state=42` para reproducibilidad y `stratify=y` para asegurar que la proporción de jugadores MVP y No MVP sea la misma en Train y en Test.

1.2. Selección de Variables y Decisión de Ingeniería de Cambio de Target

- **Target Elegido (Variable Objetivo):** `mvp_award` (1 para MVP, 0 para No MVP).
- **Motivo del Cambio de Target:** Originalmente se pensaba clasificar el resultado de la partida (`match_outcome`), pero al auditar los datos se descubrió un desbalanceo extremo: **2793 victorias vs. solo 7 derrotas**. Con un 99.75% de victorias, cualquier modelo predeciría siempre "victoria" y tendría una precisión perfecta sin aprender nada útil. Al cambiar a `mvp_award` (2561 "No" vs. 239 "Sí", un balance aproximado de 90/10), se planteó un desafío de clasificación real.
- **Variables Predictoras (Features):** Se conservaron estadísticas de juego (asesinatos, muertes, asistencias) y la variable categórica `match_outcome` (Win/Loss), ya que ganar influye directamente en ser MVP. También se incorporó la variable `performance_score` (que en la entrega anterior era el target de regresión) como una variable predictora clave.

1.3. Algoritmos Entrenados

Se entrenaron tres modelos con lógicas geométricas muy diferentes para analizar cuál se adaptaba mejor:

1. **Regresión Logística:** Un modelo lineal clásico que calcula la probabilidad de pertenecer a una clase usando una función sigmoide.
2. **K-Nearest Neighbors (KNN):** Clasifica un registro buscando los K registros más cercanos en el espacio de datos y votando por mayoría.

3. **Random Forest Classifier:** Un ensamble (grupo) de múltiples árboles de decisión que deciden de forma independiente y votan el resultado final.

2. Fase: Evaluación (Evaluation)

2.1. Resultados del Rendimiento en el Conjunto de Prueba (Test Set)

Modelo	Accuracy (Exactitud)	Precision (MVP)	Recall (Sensibilidad)	F1-Score (MVP)	ROC AUC
Regresión Logística	0.9946	0.9592	0.9792	0.9691	0.9980
K-Nearest Neighbors (K=5)	0.9517	0.8621	0.5208	0.6494	0.9995
Random Forest Classifier	1.0000	1.0000	1.0000	1.0000	1.0000

2.2. Análisis de Resultados e Hipótesis del Dataset Sintético

- **Análisis de Métricas Perfectas:** En Machine Learning, un rendimiento del 100% (1.0) suele levantar sospechas de error. Sin embargo, se estableció la hipótesis de que **este dataset es sintético y determinista**. La variable objetivo `mvp_award` fue generada mediante reglas matemáticas exactas en Python sin ruido aleatorio. Como los árboles de decisión de Random Forest y los límites de la Regresión Logística son capaces de replicar fronteras matemáticas perfectas, se logró aprender la regla exacta del dataset con total precisión.
- **La Caída de KNN:** KNN fue el modelo con menor rendimiento (F1-Score de 0.6494 y un Recall de solo 52.08% para detectar MVPs). Esto se debe a que KNN calcula

distancias espaciales y es muy sensible al desbalance de clases (al haber muchos más No-MVPs, los vecinos cercanos tienden a ser de esa clase mayoritaria) y a la alta cantidad de variables, lo que distorsiona las distancias.

2.3. Validación Cruzada (Cross-Validation) para Descartar Sobreajuste

Para asegurar que el Random Forest no se había memorizado el conjunto de datos de entrenamiento (sobreajuste), se aplicó **Validación Cruzada de 5 pliegues (5-Fold Cross-Validation)**:

- **El Concepto:** Se dividió el conjunto de entrenamiento en 5 partes. Se entrenó el modelo con 4 partes y se evaluó con la restante, rotando este proceso 5 veces para que todas las partes fueran usadas como prueba una vez.
- **El Resultado:** Se obtuvieron F1-Scores estables en cada pliegue (**1.0, 0.98, 1.0, 1.0, 0.98**) con un promedio de **0.9947**. Esto demuestra que el modelo generaliza con éxito ante datos nuevos y no está sobreajustado.

Modelo Seleccionado: Random Forest Classifier, debido a su precisión perfecta y robustez en la validación cruzada.

3. Fase: Despliegue (Deployment)

- **Serialización:** Se exportó el pipeline final entrenado a un archivo **[best_model.joblib]** (file:///Users/user/Desktop/FundamentosML/03_Modelos_Clasificacion/checkpoints/best_model.joblib) usando la librería **joblib**. Esto congela el modelo y sus pasos de escalado para que puedan ser reutilizados de inmediato.
- **Interfaz de Usuario (Streamlit):** Se creó una aplicación local interactiva (**app.py**) donde cualquier usuario puede ingresar las estadísticas de un jugador mediante sliders y menús desplegables. El modelo procesa la información y responde en tiempo real si el jugador califica como **MVP** o es un **Jugador Estándar**.